

Politechnika Świętokrzyska
Kielce University of Technology
WYDZIAŁ ELEKTROTECHNIKI, AUTOMATYKI I INFORMATYKI

Politechnika Świętokrzyska

Wydział Elektrotechniki, Automatyki i Informatyki

Rok studiów: II Semestr: III

Daniel Cieślak część praktyczna: 40% część teoretyczna: 50%

Numer albumu **092691**

Wiktoria Wójcik część praktyczna: 32.5% część teoretyczna: 30%

Numer albumu **092838**

Kamil Wyderski część praktyczna: 27.5% część teoretyczna: 20%

Numer albumu **092839**

Nazwa przedmiotu: **Programowanie w języku C2**

Temat: **Rodzinna książka telefoniczna**

Data obrony: **18.01.2023**

Kielce 2023

SPIS TREŚCI

Wstęp……..3

Cel……...3

Koncepcja projektowa……....3

1. Specyfikacja projektowa……..…4

a. Opis……...……..4
b. Uzasadnienie wyboru technologii……...…4
c. Przegląd innych podobnych rozwiązań……...…4

2. Techniczna implementacja……...…5
3. Testowanie……..……45

a. Testy funkcjonalne…….……46

4. Wnioski……..……48

a. Podsumowanie działań……..……..48
b. Możliwości rozwoju aplikacji w przyszłości…….…….48

5. Załączniki……...……48

a. Instrukcja instalacji…….……48
b. Instrukcja uruchomienia aplikacji……..……48
c. Instrukcja obsługi aplikacji…….……48
d. Wymagania aplikacji – niezbędne oprogramowanie…….……48
e. Pełen kod źródłowy aplikacji…….……48

Oświadczenie o samodzielności wykonania projektu……..……..49

WSTĘP

W dobie dzisiejszej, szybko rozwijającej się technologii stare rozwiązania pewnych problemów zostają zmienione na nowocześniejsze. Dzieje się tak w przypadku wszelakich spisów ludności, książek telefonicznych czy indeksów dla studentów, które niegdyś występowały w formie papierowej, a teraz zostały przeniesiona do, najczęściej, baz danych, do których mamy dostęp z poziomu komputera. Projekt prezentuje jedną z przykładowych aplikacji rozwiązującej temat książki telefonicznej, mianowicie jest to *Rodzinna książka telefoniczna.*

CEL

Celem projektu pt. *Rodzinna książka telefoniczna* było stworzenie odwzorowania rzeczywistego przedmiotu jakim jest książka telefoniczna w postaci programu komputerowego z interfejsem graficznym przyjaznym dla użytkownika. Jego przeznaczeniem zostało jednak katalogowanie osób z rodziny bądź bliskich znajomych osoby użytkującej, po to, aby mogła ona później wedle potrzeby wyszukać i znaleźć o nich potrzebne informacje, takie jak numer telefonu bądź na przykład adres zamieszkania.

KONCEPCJA PROJEKTOWA

Aplikacja miała być bazą danych, która pozwalała na pełną modyfikację swojej zawartości. W tym celu musiały zostać dodane opcje odpowiedzialne za dodawanie, usuwanie, a także edycję kontaktów. Kiedy jednak użytkownik posiada ich już sporą ilość, staje się również potrzebne stworzenie opcji do łatwiejszego ich przeglądania. Takimi opcjami są: wyszukanie konkretnej osoby w bazie, filtracja bazy po dowolnym jej polu, a także proste operacje sortowania malejącego lub rosnącego. Dodatkową przydatną fukcjonalnością programu, którą warto było zaimplementować, była również możliwość posiadania kilku odrębnych jego użytkowników, którzy posiadają w nim własne konta z odbrębnymi bazami danych. W tym celu musiały być dodane okna tworzenia użytkowników i ich usuwania, a także logowanie dodatkowo zabezpieczone hasłem.

1. SPECYFIKA PROJEKTOWA

Specyfikacja projektowa zawiera opis głównych i dodatkowych założeń projektu, uzasadnienie wyboru technologii użytej w aplikacji oraz przegląd innych podobnych rozwiązań.

a. Opis

Główne założenia funkcjonalności programu:

1. Dodawanie nowej osoby do książki telefonicznej
2. Usuwanie istniejącej osoby z książki telefonicznej
3. Edytowanie danych odpowiedniej osoby
4. Wyszukiwanie odpowiedniej osoby
5. Wyświetlanie całej książki telefonicznej
6. Sortowanie książki telefonicznej
7. Filtracja i kategoryzacja kontaktów z książki telefonicznej

Dodatkowe funkcje programu:

1. Logowanie do rodzinnej książki telefonicznej
2. Wyświetlanie zdjęć dla danego kontaktu z książki telefonicznej

b. Wybór technologii

Technologią wykorzystywaną do utworzenia aplikacji *Rodzinna książka telefoniczna* jest biblioteka wxWidgets ze względu na prostotę tej biblioteki oraz to, że jest to darmowe oprogramowanie działające na różnych systemach operacyjnych na przykład Windows czy Linux. Projekt został napisany w języku programowania C++ w środowisku Code::Blocks. Środowisko to zostało wybrane ze względu na jego kompatybilność z biblioteką wxWidgets (sam Code::Blocks został utworzony z pomocą tej biblioteki) oraz ze względy na fakt, iż Code::Blocks posiada kreator aplikacji w którym programista może, zamiast samemu pisać kod, ustawiając jedynie odpowiednio kontrolki stworzyć wygląd programu.

c. Inne rozwiązania

W rzeczywistości istnieje wiele podobnych rozwiązań niż wykorzystanie biblioteki wxWidgets. Do projektu można również było użyć innych bibliotek, które mają podobną ideę jak ta którą prezentuje wxWidgets, jak choćby Qt czy GTK+. Możliwości te mają jednak swoje wady, na przykład to, że framework Qt jest nie do końca darmowy a inne nie były omawiane na uczelni. Zaletą ich jest jednak fakt, że posiadają one więcej funkcji i mogą być wykorzystywane do poważniejszych projektów komercjalnych. Natomiast zamiast środowiska C::B można było wykorzystać Microsoft Visual Studio czy środowisko Qt Creator.

2. TECHNICZNA IMPLEMENTACJA

Zgodnie z koncepcją i specyfikacją projektową aplikacja pt. *Rodzinna książka telefoniczna* została stworzona w języku C++ z wykorzystaniem biblioteki graficznej wxWidgets. Niniejszy rozdział będzie opisywał implementacje jej funkcjonalności od strony kodu źródłowego, a także ukazywał zdjęcia przedstawiające efekty jej działania.

1. Kilka słów - program jest podzielony na oddzielne kody źródłowe i biblioteki zarówno te generowane przez środowisko programistyczne automatycznie przy tworzeniu nowego okna oraz te stworzone przez nas, w celu uporządkowania stworzonego kodu. Jednym z ważniejszych plików nagłówkowych jest Library.h zawierający podstawowe struktury i funkcje, które będą dalej opisywane. Co jest warte zaznaczenia dane w programie są przechowywane w wektorze.

```
vector<Person> db;
```

Tab. 1 – kod źródłowy wektora db.

Przechowuje on dane typu struktury Person, której kod zawiera się w bibliotece Library.h. Mamy w niej również zawartą identyczną strukturę, ale przechowującą typy wxString.

```
Struct Person

{

   int choose;

   int id;

   string name;

   string last_name;

   string street_name;

   unsigned int house_number;

   unsigned int postcode;

   string city;

   unsigned int phone_number;

   string relationship;

};
```

```
typedef struct PersonWx

{

    wxString choose;

    wxString id;

    wxString name;

    wxString last_name;

    wxString street_name;

    wxString house_number;

    wxString postcode;

    wxString city;

    wxString phone_number;

    wxString relationship;

}PersonWx;
```

Tab. 2 – główne struktury programu.

W programie mamy użytą jeszcze jedną istotną zmienną globalną – hv.

```
bool hv=true;
```

Tab. 3 – kod źródłowy zmiennej hv.

Jej przeznaczeniem jest kontrola stanu programu. Podczas wyszukania, filtrowania, czy sortowania zawartość głównego wektora ulega zmianie i wartość zmiennej ustawiana jest na false. Zapobiega to niekorzystym zmianom w strukturze bazy danych programu o czym szerzej w następnych podrozdziałach.

```
PersonWx convertToWx(long long int i, vector<Person> db){

    PersonWx w;

    struct Person n=db[i];

    w.id<<n.id;

    w.house_number<<n.house_number;
```

```
    w.postcode<<n.postcode;
    w.phone_number<<n.phone_number;
    wxString name(n.name);
    w.name=name;
    wxString last_name(n.last_name);
    w.last_name=last_name;
    wxString street_name(n.street_name);
    w.street_name=street_name;
    wxString city(n.city);
    w.city=city;
    wxString relationship(n.relationship);
    w.relationship=n.relationship;
    return w;
}
struct Person convertToCpp(PersonWx w){
    struct Person n;
    n.id=wxAtoi(w.id);
    n.house_number=wxAtoi(w.house_number);
    n.postcode=wxAtoi(w.postcode);
    n.phone_number=wxAtoi(w.phone_number);
    n.name=string(w.name.mb_str());
    n.last_name=string(w.last_name.mb_str());
    n.street_name=string(w.street_name.mb_str());
    n.city=string(w.city.mb_str());
    n.relationship=string(w.relationship.mb_str());
```

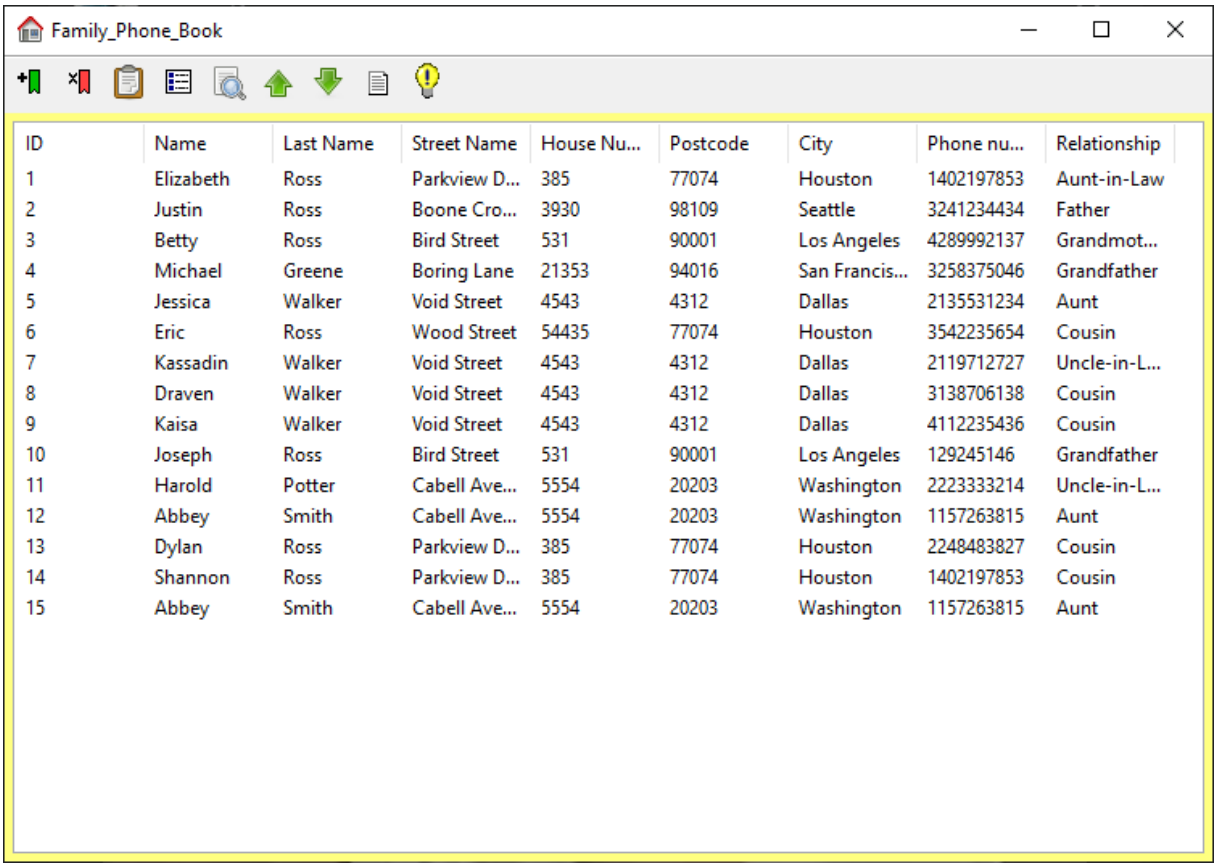
```
    return n;

}
```

Tab. 4 – funkcje convertToWx i convertToCpp .

W programie często wynika potrzeba zamiany zawartości struktury PersonWx na typy danych w strukturze Person i odwrotnie. Funkcje convertToWx i convertToCpp są za to odpowiedzialne.

2. Okno główne – stanowi ono główną część programu zawierając w centralnej swojej części zawartość całej bazy danych z podziałem na wiersze zawierające kolejne kontakty; i kolumny z polami takimi jak imię, nazwisko, miasto, numer telefonu itd. W górnej części mamy umieszczone również dodatkowo menu zawierające graficzną reprezentacje czynności możliwych do wykonania na bazie. Po najechaniu kursorem na jedną z nich pojawia się wzkazówka czego ona dotyczy, a po jej wciśnięciu zostaje wyświetlone odpowiednie okno dialogowe.



ID	Name	Last Name	Street Name	House Nu...	Postcode	City	Phone nu...	Relationship
1	Elizabeth	Ross	Parkview D...	385	77074	Houston	1402197853	Aunt-in-Law
2	Justin	Ross	Boone Cro...	3930	98109	Seattle	3241234434	Father
3	Betty	Ross	Bird Street	531	90001	Los Angeles	4289992137	Grandmot...
4	Michael	Greene	Boring Lane	21353	94016	San Francis...	3258375046	Grandfather
5	Jessica	Walker	Void Street	4543	4312	Dallas	2135531234	Aunt
6	Eric	Ross	Wood Street	54435	77074	Houston	3542235654	Cousin
7	Kassadin	Walker	Void Street	4543	4312	Dallas	2119712727	Uncle-in-L...
8	Draven	Walker	Void Street	4543	4312	Dallas	3138706138	Cousin
9	Kaisa	Walker	Void Street	4543	4312	Dallas	4112235436	Cousin
10	Joseph	Ross	Bird Street	531	90001	Los Angeles	129245146	Grandfather
11	Harold	Potter	Cabell Ave...	5554	20203	Washington	2223333214	Uncle-in-L...
12	Abbey	Smith	Cabell Ave...	5554	20203	Washington	1157263815	Aunt
13	Dylan	Ross	Parkview D...	385	77074	Houston	2248483827	Cousin
14	Shannon	Ross	Parkview D...	385	77074	Houston	1402197853	Cousin
15	Abbey	Smith	Cabell Ave...	5554	20203	Washington	1157263815	Aunt

Rys. 1 – okno główne programu.

```
if(hv){

    LoginWindow dlg(this);

    if(login){
```

```
        if(dlg.ShowModal()==wxID_OK)
        {
            path=dlg.pathtoFile;
            login=false;
        }
        else
            Close();
        }
        readDataBase(path,db);
    }



wxString line;
long counter=1;
mainWindowV->InsertColumn(0, "ID");
mainWindowV->InsertColumn(1, "Name");
mainWindowV->InsertColumn(2, "Last Name");
mainWindowV->InsertColumn(3, "Street Name");
mainWindowV->InsertColumn(4, "House Number");
mainWindowV->InsertColumn(5, "Postcode");
mainWindowV->InsertColumn(6, "City");
mainWindowV->InsertColumn(7, "Phone number");
mainWindowV->InsertColumn(8, "Relationship");
for(long long int i=(db.size()-1);i>=0;i--)
{
```

```
        wxListItem id, name, last_name, street_name, house_number, postcode, city, phone_number, relationship;

        id.SetColumn(0);

        id.SetId(counter);

        counter=mainWindowV->InsertItem(id);

        PersonWx p=convertToWx(i, db);

        mainWindowV->SetItem(counter, 0, p.id, -1);

        mainWindowV->SetItem(counter, 1, p.name, -1);

        mainWindowV->SetItem(counter, 2, p.last_name, -1);

        mainWindowV->SetItem(counter, 3, p.street_name, -1);

        mainWindowV->SetItem(counter, 4, p.house_number, -1);

        mainWindowV->SetItem(counter, 5, p.postcode, -1);

        mainWindowV->SetItem(counter, 6, p.city, -1);

        mainWindowV->SetItem(counter, 7, p.phone_number, -1);

        mainWindowV->SetItem(counter, 8, p.relationship, -1);

    }
```

Tab. 5 – część zawartości konstruktora okna głównego.

Po poprawnym zalogowaniu do bazy, wewnątrz obiektu wxListCtrl tworzone są kolumny z odpowiednimi polami oraz kolejne wiersze do których wprowadzane są wartości z głównego wektora.

3. Dodawanie nowej osoby – aby je wykonać należy w oknie głównym nacisnąć pierwszą pozycję w menu. Zostaje wtedy wywołane zdarzenie wykonujące funkcję addingButton w programie.

```
void Family_Phone_BookFrame::addingButton(wxCommandEvent& event)

{

    if(hv){

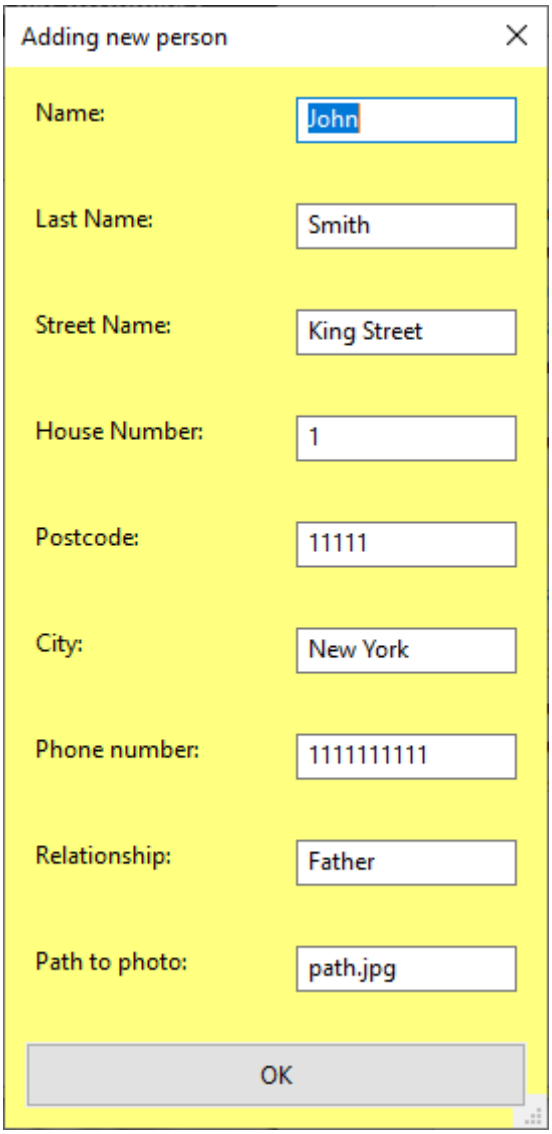
        PersonWx w;

        AddingD dlg(this);
```

```
        if(dlg.ShowModal()== wxID_OK){
            w=dlg.GetResult();
        }
        if(w.choose=="1"){
            int cntp=db.size()+1;
            struct Person n=convertToCpp(w);
            addPerson(db,path, cntp, n);
            wxMessageBox("Adding completed successfully!");
        }
    }
    hv=true;
    db.clear();
    this->Destroy();
    Family_Phone_BookFrame* mainF = new Family_Phone_BookFrame(0);
    mainF->Show();
}
```

Tab. 6 – zawartość funkcji odpowiedzialnej za dodawanie.

Funkcja sprawdza wartość zmiennej hv. Jeśli jest ona prawdą wywołuje okno dialogowe AddingD i pobiera z niego wartość zwracaną przez funkcję GetResult zapisując ją do struktury w o typie PersonWx. Następnie sprawdzane jest czy zawarte w niej pole choose ma wartość 1 i w przypadku prawdy następuje przypisanie do zmiennej cntp rozmiaru głównego wektora. Później struktura w jest konwertowana na typ struktury Person i wywoływana jest funkcja addPerson, i wyświetlany jest komunikat o udanym dodaniu nowej osoby do bazy. Na koniec zawartość wektora jest czyszczona, a okno główne zniszczone, stworzone na nowo i ukazane ponownie już z zaktualizowanymi danymi.



Rys. 2 – okno dialogowe AddingD.

Okno dialogowe AddingD składa się z pól typu wxTextCtrl, w które są wprowadzane dane przez użytkownika, a także przycisku OK. Jego naciśnięcie powoduje wywołanie funkcji AddingOK.

```
void AddingD::AddingOK(wxCommandEvent& event)

{

    PersonWx p;

    p.choose="1";

    p.name=NameV->GetValue();
```

```
    p.last_name=LastNameV->GetValue();
    p.street_name=StreetNameV->GetValue();
    p.house_number=HouseNumberV->GetValue();
    p.postcode=PostcodeV->GetValue();
    p.city=CityV->GetValue();
    p.phone_number=PhoneNumberV->GetValue();
    p.relationship=RelationshipV->GetValue();
    result=p;
    EndModal(wxID_OK);
}
```

Tab. 7 – funkcja AddingOK.

W funkcji AddingOK tworzona jest struktura typu PersonWx i kolejnym jej polom są przypisywanie wartości odpowiednich pól tekstowych z okna dialowego.

```
PersonWx GetResult() const {
        return result;
    };
```

Tab. 8 – funkcja GetResult.

Funkcja GetResult jest metodą klasy AddingD i służy tylko do zwracania wartości zmiennej prywatnej result, aby móc ją później odczytać w głównym oknie programu. Wiele okien dialogowych w programie posiada identyczną funkcję, więc później jej opis będzie pomijany.

```
void addPerson(vector<Person> &db,string f, int cntp, struct Person p)
{
    if(db.empty())
    {
    ofstream file;
    p.id=cntp;
```

```
    file.open(f,ios::out | ios::app);

    db.push_back(p);


file<<p.id<<";"<<p.name<<";"<<p.last_name<<";"<<p.street_name<<";"<<p.house
_number<<";"<<p.postcode<<";"<<p.city<<";"<<p.phone_number<<";"<<p.relatio
nship;

    file.close();

    }



    else{

    ofstream file;

    p.id=cntp;

    file.open(f,ios::out | ios::app);

    db.push_back(p);


file<<"\n"<<p.id<<";"<<p.name<<";"<<p.last_name<<";"<<p.street_name<<";"<<
p.house_number<<";"<<p.postcode<<";"<<p.city<<";"<<p.phone_number<<";"<<p
.relationship;

    file.close();

    }

}
```

Tab. 9 – funkcja addPerson.

Zadaniem funkcji addPerson z biblioteki Library.h jest dodanie nowej osoby do pliki tekstowego, a także do głównego wektora programu.

4. Usuwanie istniejącej osoby – aby je wykonać należy w oknie głównym nacisnąć druga pozycję w menu. Zostaje wtedy wywołane zdarzenie wykonujące funkcję deletingButton w programie.

```
void Family_Phone_BookFrame::deletingButton(wxCommandEvent& event)

{
```

```
    if(hv){
        PersonWx w;
        DeletingD dlg(this);
        if(dlg.ShowModal()== wxID_OK){
            w=dlg.GetResult();
        }
        if(w.choose=="1"||w.choose=="2"||w.choose=="3"){
            struct Person n=convertToCpp(w);
            if(w.choose=="1")
                deletePerson(db, n.id);
            if(w.choose=="2")
                deletePersonbyFirstLastName(db, n.name, n.last_name);
            if(w.choose=="3")
                deletePersonbyPhoneNumber(db, n.phone_number);
            saveDataBase(db, path);
            wxMessageBox("Deleting completed successfully!");
        }
    }
    hv=true;
    db.clear();
    this->Destroy();
    Family_Phone_BookFrame* mainF = new Family_Phone_BookFrame(0);
    mainF->Show();
}
```

Tab. 10 – funkcja deletingPerson.

Funkcja sprawdza wartość zmiennej hv. Jeśli jest ona prawdą wywołuje okno dialogowe DeletingD i pobiera z niego wartość zwracaną przez funkcję GetResult zapisując ją do struktury w o typie PersonWx. Później w zależności od tego co zapisane jest w jej polu choose wywoływana jest odpowiednia funkcja usuwająca. Dla przykładu deletePerson, a po niej saveDataBase. Na koniec zawartość wektora jest czyszczona, a okno główne niszczone, stworzone na nowo i ukazane ponownie już z zaktualizowanymi danymi.

```
void deletePerson(vector<Person> &db,int id)

{

   db.erase(remove_if(db.begin(),db.end(),[&](Person const &person)

   {

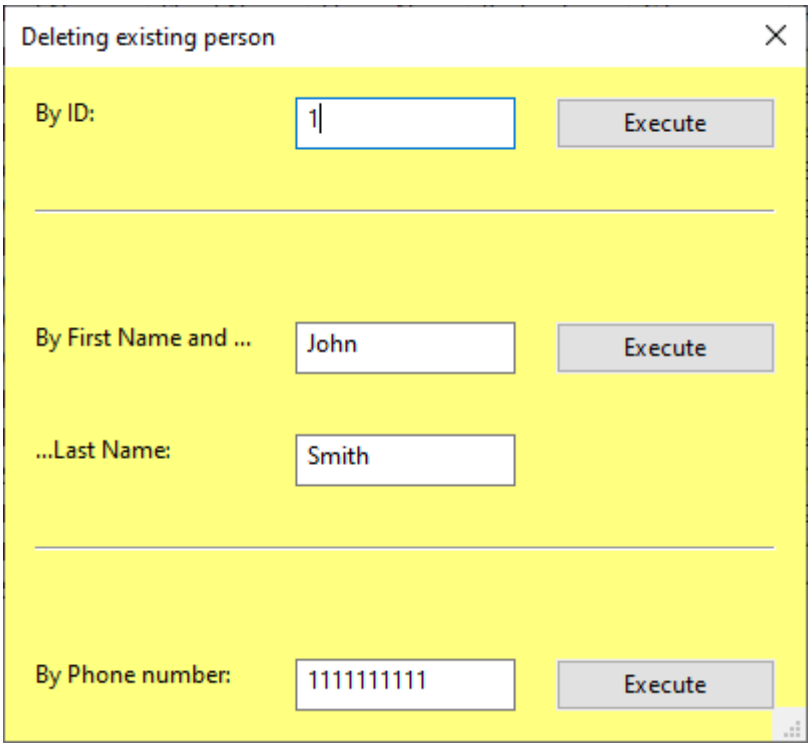
      return person.id==id;

   }),db.end());

}
```

Tab. 11 – funkcja deletePerson.

Zadaniem funkcji deletePerson z pliku nagłówkowego Library.h jest przeglądanie głównego wektora i usunięcie z niego struktury o wpisanym przez nas numerze ID. Reszta funkcji usuwających działa podobnie, więc nie będą opisywane.



Rys. 3 – okno dialogowe DeletingD.

Okno dialogowe DeletingD składa się z pól typu wxTextCtrl podzielonych na bloki, w które są wprowadzane dane przez użytkownika, a także osobnego dla każdego bloku przycisku Execute. Jego naciśnięcie powoduje wywołanie odpowiedniej funkcji usuwającej okna dla przykładu DeletingID.

```
void DeletingD::DeletingID(wxCommandEvent& event)

{

   PersonWx p;

   p.choose="1";

   p.id=IdV->GetValue();

   result=p;

   EndModal(wxID_OK);

}
```

Tab. 12 – funkcja DeletingID.

W zawartości fukcji DeletingID mamy zadeklarowanie struktury p typu PersonWx, a także przypisanie jej polu choose wartości 1. Później z pola tekstowego pobierane jest ID i zapisywane do pola id owej struktury. Potem następuje przypisanie zmiennej result jej wartości.

5. Edycja istniejącej osoby – aby ją wykonać należy w oknie głównym nacisnąć trzecią pozycję w menu. Zostaje wtedy wywołane zdarzenie wykonujące funkcję editingButton w programie.

```
void Family_Phone_BookFrame::editingButton(wxCommandEvent& event)

{

   if(hv){

      PersonWx w;

      EditingD dlg(this);

      if(dlg.ShowModal()== wxID_OK){

         w=dlg.GetResult();

      }
```

```
if(w.choose=="1"||w.choose=="2"||w.choose=="3"||w.choose=="4"||w.choose=="5"||
|w.choose=="6"||w.choose=="7"||w.choose=="8"||w.choose=="9"){
        struct Person n=convertToCpp(w);
        if(w.choose=="1")
           editPersonbyNAME(db, n.id , n.name);
        if(w.choose=="2")
           editPersonbyLAST_NAME(db, n.id , n.last_name);
        if(w.choose=="3")
           editPersonbySTREET_NAME(db, n.id , n.street_name);
        if(w.choose=="4")
           editPersonbyHOUSE_NUMBER(db, n.id , n.house_number);
        if(w.choose=="5")
           editPersonbyPOSTCODE(db, n.id , n.postcode);
        if(w.choose=="6")
           editPersonbyCITY(db, n.id , n.city);
        if(w.choose=="7")
           editPersonbyRELATIONSHIP(db, n.id , n.relationship);
        if(w.choose=="8")
           editPersonbyPHONE_NUMBER(db, n.id , n.phone_number);
        if(w.choose=="9")
           editPersonbyPHOTO_PATH(db, n.id, n.photo_path);
        saveDataBase(db,path);
        wxMessageBox("Editing completed successfully!");
      }
   }
```

```
    hv=true;
    db.clear();
    this->Destroy();
    Family_Phone_BookFrame* mainF = new Family_Phone_BookFrame(0);
    mainF->Show();
}
```

Tab. 13 – funkcja editingButton.

Działanie funkcji editingButton nie jest zaskoczeniem. Na początku sprawdza wartość zmiennej hv. Jeśli jest ona prawdą wywołuje okno dialogowe EditingD i pobiera z niego wartość zwracaną przez funkcję GetResult zapisując ją do struktury w o typie PersonWx. Później w zależności od tego co zapisane jest w jej polu choose wywoływana jest odpowiednia funkcja edytująca. Dla przykładu editPersonbyNAME, a po niej saveDataBase. Na koniec zawartość wektora jest czyszczona, a okno główne niszczone, stworzone na nowo i ukazane ponownie już z zaktualizowanymi danymi.

```
void editPersonbyNAME(vector<Person> &db,int ID, string NEW_NAME)
{
    for(unsigned long long int i=0;i<db.size();++i)
    {
        if(db[i].id==ID)
        {
            db[i].name=NEW_NAME;
        }
    }
}
```

Tab. 14 – funkcja editPersonbyNAME.

Funkcja editPersonbyNAME z pliku nagłówkowego Library.h przegląda główny wektor i jeśli trafi na strukturę o szukanym numerze ID to jej polu name przypisana zostaje nowa wpisana przez użytkownika wartość. Reszta funkcji edytujących działa podobnie, więc nie będą opisywane.



Rys. 4 – okno dialogowe EditingD.

Okno dialogowe EditingD dzieli się na dwie sekcje. W pierwszej, w polu tekstowym, wpisujemy numer ID użytkownika, którego chcemy edytować. W drugiej wybieramy pole, jakie chemy edytować, poprzez wpisanie wartości w dane pole tekstowe i naciśnięcie przycisku Execute. Jego naciśnięcie powoduje wywołanie odpowiedniej funkcji edytującej okna dla przykładu EditingName.

```
void EditingD::EditingName(wxCommandEvent& event)

{
```

```
    PersonWx p;
    p.choose="1";
    p.id=IdV->GetValue();
    p.name=NameV->GetValue();
    result=p;
    EndModal(wxID_OK);
}
```

Tab. 15 – funkcja EditingName.

W metodzie EditingName następuje zadeklarowanie struktury p typu PersonWx, a także przypisanie jej polu choose wartości 1. Później z pola tekstowego pobierane jest ID i imię, i zapisywane do odpowiednich pól owej struktury. Potem następuje przypisanie zmiennej result jej wartości.

6. Filtracja osób w bazie – aby ją wykonać należy w oknie głównym nacisnąć czwartą pozycję w menu. Zostaje wtedy wywołane zdarzenie wykonujące funkcję filteringButton w programie.

```
void Family_Phone_BookFrame::filteringButton(wxCommandEvent& event)
{
    if(hv){
        PersonWx w;
        FilteringD dlg(this);
        if(dlg.ShowModal()== wxID_OK){
            w=dlg.GetResult();
        }

if(w.choose=="1"||w.choose=="2"||w.choose=="3"||w.choose=="4"||w.choose=="5"||
|w.choose=="6"||w.choose=="7"||w.choose=="8"||w.choose=="9"){
            if(w.choose=="1")
                filterPersonbyID(w, db);
            if(w.choose=="2")
```

```
            filterPersonbyNAME(w, db);
        if(w.choose=="3")
            filterPersonbyLASTNAME(w, db);
        if(w.choose=="4")
            filterPersonbySTREETNAME(w, db);
        if(w.choose=="5")
            filterPersonbyHOUSENUMBER(w, db);
        if(w.choose=="6")
            filterPersonbyPOSTCODE(w, db);
        if(w.choose=="7")
            filterPersonbyCITY(w, db);
        if(w.choose=="8")
            filterPersonbyPHONENUMBER(w, db);
        if(w.choose=="9")
            filterPersonbyRELATIONSHIP(w, db);
        wxMessageBox("Filtering completed successfully!");
        hv=false;
        this->Destroy();
        Family_Phone_BookFrame* mainF = new Family_Phone_BookFrame(0);
        mainF->Show();
    }
    else{
        db.clear();
        this->Destroy();
        Family_Phone_BookFrame* mainF = new Family_Phone_BookFrame(0);
```

```
            mainF->Show();
        }
    }
    else{
        db.clear();
        hv=true;
        this->Destroy();
        Family_Phone_BookFrame* mainF = new Family_Phone_BookFrame(0);
        mainF->Show();
    }
}
```

Tab. 16 – funkcja filteringButton.

Metoda filteringButton na początku sprawdza wartość zmiennej hv. Jeśli jest ona prawdą wywołuje okno dialogowe FilteringD i pobiera z niego wartość zwracaną przez funkcję GetResult zapisując ją do struktury w o typie PersonWx. Później w zależności od tego co zapisane jest w jej polu choose wywoływana jest odpowiednia funkcja filtrująca. Dla przykładu filterPersonbyID, a po niej zmiennej hv jest ustawiana wartość false. Na koniec zawartość wektora jest czyszczona, a okno główne niszczone, stworzone na nowo i ukazane ponownie już z zaktualizowanymi danymi.

```
void filterPersonbyID(PersonWx w, vector<Person> &db)
{
    struct Person n=convertToCpp(w);
    vector<Person> hp;
    for(unsigned long long int i=0;i<db.size();++i)
    {
        if(db[i].id==n.id)
            hp.push_back(db[i]);
    }
```

```
    db.clear();

    for(unsigned long long int i=0;i<hp.size();++i)

    {

        db.push_back(hp[i]);

    }

}
```

Tab. 17 – funkcja filterPersonbyID.

Funkcja filterPersonbyID z bibioteki Library.h przegląda główny wektor i jeśli trafi na strukturę o szukanym numerze ID to dodaje ją do tymczasowego wektora hp. Później czyści główny wektor i dodaje do niego wartości ze zmiennej hp. Reszta funkcji filtrujących działa podobnie, więc nie będą opisywane.



Rys. 5 – okno dialogowe FilteringD.

W oknie dialogowym FilteringD użytkownik wybiera pole, jakie chce edytować, poprzez wpisanie wartości w dane pole tekstowe i naciśnięcie przycisku Execute. Jego naciśnięcie powoduje wywołanie odpowiedniej funkcji edytującej okna dla przykładu FilteringID.

```
void FilteringD::FilteringID(wxCommandEvent& event)

{

   PersonWx p;

   p.choose="1";

   p.id=IdV->GetValue();

   result=p;

   EndModal(wxID_OK);

}
```

Tab. 18 – funkcja filterPersonbyID.

W metodzie EditingName następuje zadeklarowanie struktury p typu PersonWx, a także przypisanie jej polu choose wartości 1. Później z pola tekstowego pobierane jest ID i zapisywane do odpowiedniego pola owej struktury. Potem następuje przypisanie zmiennej result jej wartości.

7. Wyszukanie konkretnej osoby w bazie – aby to wykonać należy w oknie głównym nacisnąć piątą pozycję w menu. Zostaje wtedy wywołane zdarzenie wykonujące funkcję searchingButton w programie.

```
void Family_Phone_BookFrame::searchingButton(wxCommandEvent& event)

{

   if(hv){

      PersonWx w;

      SearchingD dlg(this);

      if(dlg.ShowModal()== wxID_OK){

         w=dlg.GetResult();

      }

      if(w.choose=="1"){
```

```
            searchPerson(w, db);
            wxMessageBox("Searching completed successfully!");
            hv=false;
            this->Destroy();
            Family_Phone_BookFrame* mainF = new Family_Phone_BookFrame(0);
            mainF->Show();
        }
        else{
            db.clear();
            this->Destroy();
            Family_Phone_BookFrame* mainF = new Family_Phone_BookFrame(0);
            mainF->Show();
        }
    }
    else{
        db.clear();
        hv=true;
        this->Destroy();
        Family_Phone_BookFrame* mainF = new Family_Phone_BookFrame(0);
        mainF->Show();
    }
}
```

Tab. 19 – funkcja filterPersonbyID.

Wewnątrz metody filterPersonbyID sprawdzana jest wartość zmiennej hv. Jeśli jest ona prawdą wywołuje okno dialogowe FilteringD i pobiera z niego wartość zwracaną przez funkcję GetResult zapisując ją do struktury w o typie PersonWx. Później jeśli w polu choose jest wartość 1 jest wykonywana funkcja searchPerson, a po niej zmiennej hv jest ustawiana wartość

false. Na koniec zawartość wektora jest czyszczona, a okno główne niszczone, stworzone na nowo i ukazane ponownie już z zaktualizowanymi danymi.

```
void searchPerson(PersonWx w, vector<Person> &db)

{

   struct Person n=convertToCpp(w);

   vector<Person> hp;

   for(unsigned long long int i=0;i<db.size();++i)

   {

      if((db[i].name==n.name)&&(db[i].last_name==n.last_name))

         hp.push_back(db[i]);

   }

   db.clear();

   for(unsigned long long int i=0;i<hp.size();++i)

   {

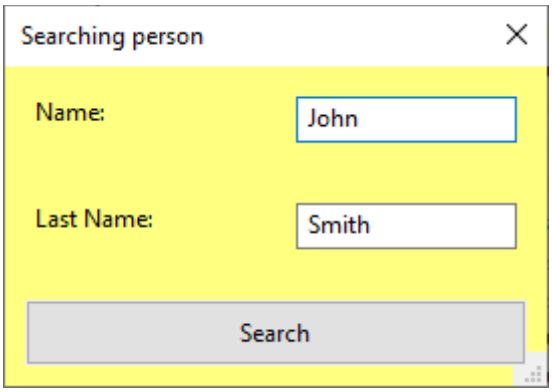
         db.push_back(hp[i]);

   }

}
```

Tab. 20 – funkcja searchPerson.

Funkcja searchPerson z bibioteki Library.h przegląda główny wektor i jeśli trafi na strukturę o szukanym imieniu i nazwisku to dodaje ją do tymczasowego wektora hp. Później czyści główny wektor i dodaje do niego wartości ze zmiennej hp.



Rys. 6 – okno dialogowe SearchingD.

W oknie dialogowym SearchingD należy wpisać imię i nazwisko szukanej osoby i zatwierdzić je przyciskiem Search. Jego naciśnięcie spowoduje wywołanie funkcji szukającej okna.

```
void SearchingD::SearchingOK(wxCommandEvent& event)

{

   PersonWx p;

   p.choose="1";

   p.name=NameV->GetValue();

   p.last_name=LastNameV->GetValue();

   result=p;

   EndModal(wxID_OK);

}
```

Tab. 21 – funkcja SearchingD.

Zadaniem funkcji SearchingD jest zadeklarowanie struktury p typu PersonWx, a także przypisanie jej polu choose wartości 1. Później z pól tekstowego pobierane jest imię i nazwisko i zapisywane do odpowiedniego pól owej struktury. Potem następuje przypisanie zmiennej result jej wartości.

8. Sortowanie osób po dowolnym polu – aby wykonać sortowanie rosnąco należy nacisnąć szóstą pozycję w menu, a by posortować dane malejąco siódmą. Zostaje wtedy wywołane zdarzenie wykonujące funkcję odpowiednio sortingASCButton lub sortingDESCButton w programie. Są one na tyle podobne, że dalej będzie opisywana tylko ta pierwsza.

```
void Family_Phone_BookFrame::sortingASCButton(wxCommandEvent& event)

{

   if(hv){

      short chs;

      SortingD dlg(this);

      if(dlg.ShowModal()== wxID_OK){

         chs=dlg.GetResult();
```

```
    }
    if(chs>0&&chs<10){
        if(chs==1)
            sortByIDAsc(db);
        if(chs==2)
            sortByNAMEAsc(db);
        if(chs==3)
            sortByLASTNAMEAsc(db);
        if(chs==4)
            sortBySTREETNAMEAsc(db);
        if(chs==5)
            sortByHOUSENUMBERAsc(db);
        if(chs==6)
            sortByPOSTCODEAsc(db);
        if(chs==7)
            sortByCITYAsc(db);
        if(chs==8)
            sortByPHONENUMBERAsc(db);
        if(chs==9)
            sortByRELATIONSHIPAsc(db);
        wxMessageBox("Sorting completed successfully!");
        hv=false;
        this->Destroy();
        Family_Phone_BookFrame* mainF = new Family_Phone_BookFrame(0);
        mainF->Show();
```

```
        }
        else{
            db.clear();
            this->Destroy();
            Family_Phone_BookFrame* mainF = new Family_Phone_BookFrame(0);
            mainF->Show();
        }
    }
    else{
        db.clear();
        hv=true;
        this->Destroy();
        Family_Phone_BookFrame* mainF = new Family_Phone_BookFrame(0);
        mainF->Show();
    }
}
```

Tab. 22 – funkcja SearchingD.

Metoda SearchingD na początku sprawdza wartość zmiennej hv. Jeśli jest ona prawdą wywołuje okno dialogowe FilteringD i pobiera z niego wartość zwracaną przez funkcję GetResult zapisując ją do zmiennej chs o typie short. Później w zależności od tego jaką chs ma wartość wywoływana jest odpowiednia funkcja sortująca. Dla przykładu sortByIDAsc, a po niej zmiennej hv jest ustawiana wartość false. Na koniec zawartość wektora jest czyszczona, a okno główne niszczone, stworzone na nowo i ukazane ponownie już z zaktualizowanymi danymi.

```
void sortByIDAsc(vector<Person> &db)
{
    vector<Person> pom;
    for(unsigned long long int j=0;j<db.size();++j)
```

```
        pom.push_back(db[j]);
    bool bChange;
    do{
        bChange=false;
        for(unsigned long long int i=0;i<db.size()-1;++i)
        {
            if(pom[i].id>pom[i+1].id)
            {
                struct Person tmp;
                tmp=pom[i];
                pom[i]=pom[i+1];
                pom[i+1]=tmp;
                bChange=true;
            }
        }
    }while(bChange);
    db.clear();
    for(unsigned long long int i=0;i<pom.size();++i)
    {
            db.push_back(pom[i]);
    }
}
```

Tab. 23 – funkcja sortByIDAsc.

Funkcja searchPerson z pliku nagłówkowego Library.h przegląda główny wektor i zapisuje posortowane wartości ID do wektora tymczasowego pom. Później czyści główny wektor i dodaje do niego wartości ze zmiennej pom.



Rys. 7 – okno dialogowe SortingD.

W oknie dialogowym FilteringD użytkownik wybiera pole poprzez naciśnięcie przycisku Execute. Spowoduje to wywołanie odpowiedniej funkcji sortującej okna dla przykładu SortingID.

```
void SortingD::SortingID(wxCommandEvent& event)

{

   result=1;

   EndModal(wxID_OK);

}
```

Tab. 24 – funkcja SortingID.

Metoda SortingID jedyne co robi to przypisuje zmiennej result wartość 1.

9. Informacje o autorach – aby wyświetlić informacje o autorach należy nacisnąć ostatnią pozycję w menu. Zostaje wtedy wywołane zdarzenie wykonujące funkcję moreButton. Jej jedynym zadaniem jest wyświetlenie obiektu typu wxMessageBox.

```
void Family_Phone_BookFrame::moreButton(wxCommandEvent& event)

{

   wxMessageBox("Authors: Daniel Cieslak, Wiktoria Wojcik, Kamil Wyderski");

}
```

Tab. 25 – funkcja moreButton.

10. Funkcja saveDataBase – nieopisawana wcześniej funkcja opiowiedzialna za zapisywanie bazy danych do pliku. Była ona wywoływana po operacjach usuwania i edytowania bazy.

```
void saveDataBase(vector<Person> &db,string f)

{

   ofstream file;

   file.open(f,ios::out | ios::trunc);

   for(unsigned long long int i=0;i<db.size();++i)

   {

      if(i<(db.size()-1))


file<<db[i].id<<";"<<db[i].name<<";"<<db[i].last_name<<";"<<db[i].street_name<
<";"<<db[i].house_number<<";"<<db[i].postcode<<";"<<db[i].city<<";"<<db[i].ph
one_number<<";"<<db[i].relationship<<";"<<db[i].photo_path<<"\n";

      if(i==(db.size()-1))


file<<db[i].id<<";"<<db[i].name<<";"<<db[i].last_name<<";"<<db[i].street_name<
<";"<<db[i].house_number<<";"<<db[i].postcode<<";"<<db[i].city<<";"<<db[i].ph
one_number<<";"<<db[i].relationship<<";"<<db[i].photo_path;

   }

   file.close();

}
```

Tab. 26 – funkcja saveDataBase.

11. Szerzej o hv – wspomina wcześciej zmienna hv będąca skrótem od helping value jest pomocniczą wartością typu prawda/fałsz bez której program nie działałby poprawnie. Jest ona ważna, ponieważ jak można było zauważyć w operacjach filtrowania, wyszukiwania, sortowania zawartość głównego wektora jest zastępowana innymi danymi. Dzieje się tak, aby okno główne mogło zostać zniszczone i stworzone na nowo, ale bez wczytania wartości z pliku, tylko z wektora przez co pozbyto się konieczości tworzenia nowego okna do każdej operacji. Wszystkie one są widoczne na głównym. Dodatkowo naciścięcie dowolnego przycisku w menu powoduje przypisanie wartości hv prawdy przez co w oknie z powrotem pokazują się wartości z pliku tekstowego.

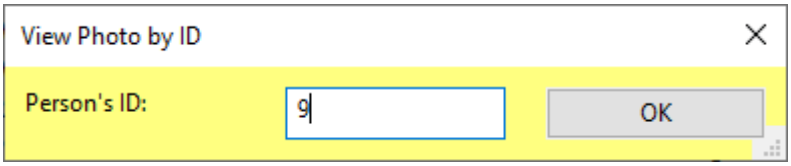
12. Wyświetlanie zdjęcia istniejącej osoby – do wyświetlania zdjęcia osoby wykorzystywane są dwa okna dialogowe, jedno do wczytywania id osoby, której zdjęcie chcemy wyświetlić, drugi do wyświetlenia tego zdjęcia. Aby wyświetlić pierwszy dialog w menu należy nacisnąć ostatni przycisk. Zostaje wywołane wtedy zdarzenie wykonujące funkcję viewphotoButton w programie.

```
void Family_Phone_BookFrame::viewphotoButton(wxCommandEvent& event)
{
   if(hv){
      PersonWx w;
      ViewingD dlg(this);
      if(dlg.ShowModal()==wxID_OK){
         w=dlg.GetResult();
         for(unsigned long long int lic=1;db.size()+1;lic++){
         wxString temp_lic;
         int tmp;
         tmp=wxAtoi(w.id);
         temp_lic<<lic;
         if(tmp<=0||tmp>db.size()+1){
```

```
            wxMessageBox("Wrong identifier!");
            break;
        }
        if(w.id==temp_lic){
            wxString pp;
            pp<<db[lic-1].photo_path;
            w.photo_path=pp;
            PhotoD dlg2(w,this);
            dlg2.ShowModal();
            break;
        }
        }

    }
    }
  hv=true;
  db.clear();
  this->Destroy();
  Family_Phone_BookFrame* mainF = new Family_Phone_BookFrame(0);
  mainF->Show();
}
```

Tab. 27 – funkcja viewphotoButton.



Rys 8. – okno dialogowe ViewingD

Okno dialogowe posiada jedną funkcję, która zwraca obiekt o typie PersonWx zawierający id, które wczytał użytkownik.

```
void ViewingD::OnButton1Click(wxCommandEvent& event)

{

        PersonWx p;

        p.choose="1";

        p.id=TextCtrl1->GetValue();

        result=p;

        EndModal(wxID_OK);

}
```

Tab 28. – funkcja zwracająca osobę o danym id

Funkcja sprawdza wartość zmiennej hv. Jeśli jest ona prawdą wywołuje okno dialogowe ViewingD i pobiera z niego wartość zwracaną przez funkcję GetResult() zapisując ją do struktury w o typie PersonWx. Później w zależności od tego co zapisane jest w jej polu id wywoływana jest odpowiednie okno dialogowe PhotoD które w swoim konstruktorze pobiera strukturę o typie PersonWx.



Rys 9. – przykładowy wynik okna dialogowego PhotoD

```
PhotoD::PhotoD(PersonWx per, wxWindow* parent,wxWindowID id,const wxPoint& pos,const wxSize& size)

{

        //…

        photo=wxImage(per.photo_path);

        photo.Rescale(512,512);

        StaticBitmap1->SetBitmap(photo);
```

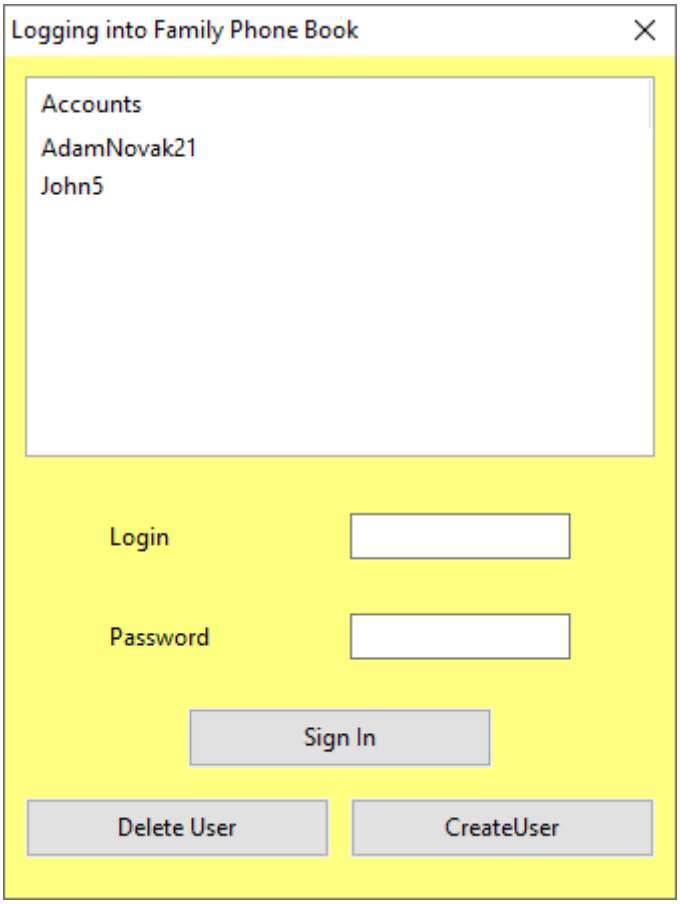
```
}
```

Tab 29. – ustawianie bitmapy i skalowanie zdjęcia

Okno dialogowe PhotoD zawiera statyczną bitmapę, która jest ustawiana przy pomocy klasy wxImage, do której obiektu o nazwie *photo* jest wczytywana ścieżka do pliku.

Logowanie

13. Ekran logowania – odpowiada za obsługę takich funkcjonalności jak: wyświetlenie loginu istniejących użytkowników zapisanych w bazie, tworzenie nowego użytkownika, usuwanie istniejącego użytkownika oraz logowanie do głównej części projektu czyli książki telefonicznej**.**



Rys. 10 – okno dialogowe logownia.

14. **Logownie-** aby się zalogować należy podać login oraz hasło do już istniejącego konta. W przypadku gdy dane są nie poprawne zostanie wyświetlony komunikat informujący o nie poprawnym wprowadzeniu danych. Jest to możliwe dzięki sprawdzeniu czy wpisane znajdują się w bazie danych. W momencie wpisania poprawnych danych oraz naciśnięciu przycisku odpowiadającego za zalogowanie do zmiennej pathtoFile zwracana jest nazwa pliku w którym zapisywane są dane użytkownika.

```
void LoginWindow::SignInButton(wxCommandEvent& event)

{

   wxString loginW(LoginValue->GetValue());

   wxString passwordW(PasswordValue->GetValue());

   string login = string(loginW.mb_str());

   string password = string(passwordW.mb_str());



   for(unsigned int i=0;i<ACCdb.size();i++)

   {

     if(ACCdb[i].login == login && ACCdb[i].password == password)

     {

         wxMessageBox("Welcome!");

         error = false;

         pathtoFile = ACCdb[i].file;

         EndModal(wxID_OK);

     }

   }

   if(error)

   {

       wxMessageBox("Incorrect Values");

   }

}
```

Tab. 30 – funkcja SignInButton.

Funkcja *SignInButton*, która służy do obsługi logowania. W przypadku gdy wprowadzone dane są nie poprawne zostanie wyświetlony komunikat informujący o nie poprawnym wprowadzeniu danych. Jest to możliwe dzięki sprawdzeniu czy wpisane znajdują się w bazie danych. W momencie wpisania poprawnych danych oraz naciśnięciu przycisku odpowiadającego za zalogowanie do zmiennej *pathtoFile* zwracana jest nazwa pliku w którym zapisywane są dane użytkownika.

```
bool ck = true;

bool error = true;
```

Tab. 31 – zmienne globalne.

W programie zostały użyte dodatkowe zmienne globalne. Zmienna ck odpowiada za warunek czy należy wczytać konta z pliku. Zmienna error przechowuje warunek który jest potrzeby by sprawdzić czy należy wyświetlić okno z błędem.

15. **Dodawanie użytkownika –** aby je uruchomić należy w ekranie logowanie nacisnąć przycisk *CreateUser*. Spowoduje to pojawienia się nowego okna dialogowego które służy do wprowadzania loginu oraz hasła dla nowego użytkownika.

```
void LoginWindow::CreateUserButton(wxCommandEvent& event)

{

   CreateUserD dlg(this);

   int res = dlg.ShowModal();

   ACCdb.clear();

   string accPath = "accounts.txt";

   readAccount(accPath,ACCdb);

   UsersListV->ClearAll();

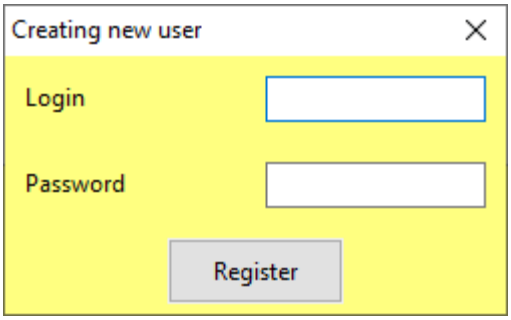
   UsersListV->InsertColumn(0, "Login");

   UsersListV->SetColumnWidth(0,250);

   long counter=1;
```

```
    for(long long int i=(ACCdb.size()-1);i>=0;i--)
    {
        counter=UsersListV->InsertItem(0,"1");
        AccountWx p = UserConvertToWx(i, ACCdb);
        UsersListV->SetItem(counter, 0, p.login, -1);
    }
}
```

Tab. 32 – funkcja CreateUserButton .



Rys. 11 – okno dialogowe tworzenia użytkownika.

Funkcja *CreateUserButton* odpowiada za utworzenie nowego okna dialogowego. W momencie gdy tworzenie nowego konta przebiegnie poprawnie okno to jest zamykane. Nowo utworzony użytkownik jest dodawany do bazy oraz zapisywany do pliku, który przechowuje loginy hasła oraz ścieżki dla poszczególnych kont. Następnie do listy w ekranie głównym logowania dodawane są loginy kont, które istnieją w bazie.

```
void CreateUserD::OnRegisterButton(wxCommandEvent& event)
{

    wxString loginW(LoginValue2->GetValue());
    wxString passwordW(PasswordValue2->GetValue());
```

```
    string login = string(loginW.mb_str());
    string password = string(passwordW.mb_str());


    if(login.empty() || password.empty())
    {
        wxMessageBox("Incorrect Values");
    }
    else
    {
        string f = "accounts.txt";
        ofstream file;
        file.open(f,ios::out | ios::app);
        Account tmp;
        string namefile = login +".txt";
        file<<"\n"<<login<<";"<<password<<";"<<namefile;
        ofstream CreateFile(namefile);
        CreateFile.close();
        EndModal(wxID_OK);
    }
}
```

Tab. 33 – funkcja OnRegisterButton.

Funkcja OnRegisterButton odpowiada za pobranie wartości które znajdują się w polach służących do wpisywania danych. W przypadku gdy, któreś z tych pól jest puste zostaje wyświetlony komunikat informujący o podaniu niepoprawnych wartości. Kiedy dane są poprawnie wpisane zostają dodane do pliku a następnie zostaje utworzony plik, który będzie przechowywał kontakty dla danego użytkownika.

16. **Usuwanie użytkownika** aby je uruchomić należy w ekranie logowanie nacisnąć przycisk Delete User. Spowoduje to pojawienia się nowego okna dialogowego które służy do wprowadzania loginu oraz hasła dla konta, które ma zostać usunięte.

```
void LoginWindow::DeleteUserButton(wxCommandEvent& event)

{

   DeleteUserD dlg(this);

   int res = dlg.ShowModal();



   ACCdb.clear();

   string accPath = "accounts.txt";

   readAccount(accPath,ACCdb);

   UsersListV->ClearAll();

   UsersListV->InsertColumn(0, "Login");

   UsersListV->SetColumnWidth(0,250);

   long counter=1;



   for(long long int i=(ACCdb.size()-1);i>=0;i--)

   {

      counter=UsersListV->InsertItem(0,"1");

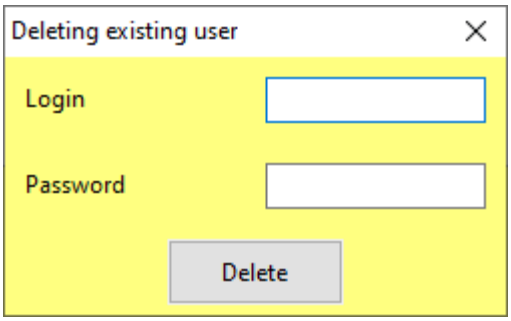
      AccountWx p = UserConvertToWx(i, ACCdb);

      UsersListV->SetItem(counter, 0, p.login, -1);

   }

}
```

Tab.34 – funkcja DeleteUserButton.



Rys. 12 – okno dialogowe usuwanai użytkownika.

```
void deleteAccount(vector<Account> &acc, string login, string password)
{


   for(unsigned long long int i=0;i<acc.size();++i)
   {
      if(acc[i].login == login && acc[i].password == password)
      {
         acc.erase(remove_if(acc.begin(),acc.end(),[&](Account const &account)
         {
            return account.login==login;
         }),
         acc.end());
      }
   }
}
```

Tab. 35 – funkcja deleteAccount.

Zadaniem funkcji deleteAccount z biblioteki Library.h jest usunięcie użytkownika z bazy danych która przechowywana jest w wektorze.

```
void DeleteUserD::DeleteButton(wxCommandEvent& event)

{

    readAccount(accPath,AccDel);

    wxString loginW(LoginValue->GetValue());

    wxString passwordW(PasswordValue->GetValue());

    string login = string(loginW.mb_str());

    string password = string(passwordW.mb_str());

    deleteAccount(AccDel, login, password);

    SaveAccDB(AccDel,accPath);

    EndModal(wxID_OK);

}
```

Tab. 36 – funkcja DeleteButton.

Zadaniem funkcji DeleteButton jest pobranie wartości z pól tekstowych a także przekazanie tychże wartości jako parametry funkcji deleteAccount, która to usuwa konto o podanych danych. Po usunięciu konta baza jest zapisywana a okno dialogowe jest zamykane.

3. TESTOWANIE

Testowanie projektu zostało wykonane w dwóch etapach. Pierwszy z nich polegał na testach funkcjonalności projektu wykonywanych przez osoby postronne, a drugi polegał na testach kodu przez osoby z grupy.

a. Testy funkcjonalne

Testy funkcjonalne pierwszej osoby spoza grupy wykonującej projekt:

Pierwszy tester: Weronika Wójcik

Ocena aplikacji po testach:

⇒ Pierwsze odczucia po odpaleniu aplikacji były pozytywne, osoba oceniająca stwierdziła, że logowanie jest czytelne i ma ładną oprawę graficzną.
⇒ Następnie osoba, po zalogowaniu się, miała przetestować działanie poszczególnych funkcji programu. Tester był zadowolony z faktu, że elementy aplikacji mogą mieć zmienianą wielkość, ale zauważył również, że nagłówki od atrybutów osoby (ID, imię itp.) powinny być od razu większe, tak aby użytkownik nie musiał tego robić.
⇒ Tester zaczął od testowania funkcji do wyświetlania zdjęć i był szczerze zadowolony z jej działania. Później osoba stwierdziła, że dobrze, iż program zawiera przycisk wyświetlający autorów pracy. Po sprawdzeniu funkcji sortujących osoba testująca program, nie wiedziała co się zmieniło – było to mało czytelne. Tester był pozytywnie zaskoczony z faktu, iż aplikacja posiada sortowanie malejące i rosnące, oraz z tego, że funkcje od edycji danych osoby oraz filtracji i kategoryzacji osoby posiadają wiele elementów (kategorii), po których można wykonać daną czynność.
⇒ Po skończonych testach funkcjonalności osoba testująca stwierdziła, że ogólny wygląd aplikacji jest spójny, posiada dobrą czcionkę. Pewną przeszkodą, którą tester napotkał na swojej drodze jest włączanie i wyłączanie okna aplikacji po danych funkcjach, jednakże w odczuciu testera nie jest to uciążliwe. Osoba testująca stwierdziła również, że aby korzystać z książki telefonicznej należy znać język angielski, gdyż w takim języku napisana jest cała książka.

Testy funkcjonalne drugiej osoby spoza grupy wykonującej projekt:

Drugi tester: Dominika Wyderska

Ocena aplikacji po testach:

⇒ Pierwsze odczucia osoby testującej program były dobre. Tester stwierdził, że interfejs jest przejrzysty oraz miły dla oka.
⇒ Na ekranie logowania tester pochwalił fakt listy użytkowników, którzy wpisani są w bazę. Osoba wskazała na brak możliwość odzyskiwania/przypominania hasła.
⇒ Po zalogowaniu, tester zaczął korzystać z funkcji, które są dostępne w aplikacji. Pierwszą z nich było dodanie nowego kontaktu do książki przy której tester stwierdził, że uciążliwe jest usuwanie wartości, które znajdują się w polach służących do podania wartości dodawanej osoby. W reszcie dostępnych opcji osoba testująca była mile zaskoczona ilością kategorii, po których można np. sortować osoby, oraz nie zauważyła większych błędów a także niedociągnięć w tychże kategoriach. Możliwość wyświetlenie zdjęcia danej osoby z bazy także była pozytywnie odebrana przez testującego.

⇒ Po zakończonych testach osoba, która testowała aplikacje, wyraziła pozytywna opinię na temat przebiegu testu oraz całości aplikacji, a także stwierdziła, że widzi potencjał drzemiący w projekcie. Osoba podała kilka ważnych zmian, które można dokonać, by poprawić aplikacje. Pierwszym jest usunięcia resetowania okna aplikacji użyciu funkcji, ponieważ nie są zapamiętywane rozmiary wcześniej ustawionego okna. Następnym dobrym pomysłem byłoby dodanie możliwości wyboru języka aplikacji, gdyż osoby, które nie znają języka angielskiego, mogą mieć problem z korzystania z aplikacji. Ostatnią rzeczą jest dodanie możliwość potwierdzania niektórych akcji przyciskiem ENTER.

Testy funkcjonalne trzeciej osoby spoza grupy wykonującej projekt:

Trzeci tester: Dawid Cieślak

Ocena aplikacji po testach:

⇒ Pierwsze zetknięcie z programem było pozytywne. Poczas logowania osoba testująca zwróciła uwagę na brak zatwierdzania opcji klawiszem ENTER, ale w jej odczuciu nie było to problemem, a także na brak opcji pokazania hasła.
⇒ Później osoba naprzemian dodawała i usuwała kontakty celem zepsucia programu. Nie udało jej się jednak tego dokonać.
⇒ Wielokrotne klikanie na przemian przypadkowych opcji również nie naruszyło stabilności aplikacji.
⇒ Skalowalność aplikacji i jej przejrzystość zostały wymienione jako jej zalety.
⇒ Na koniec tester był zadowolony z możliwości wyświetlania zdjęć.
⇒ Po teście osoba stwierdziła, że program jest ciekawy i przydatny.

Testy funkcjonalne pierwszej osoby z grupy wykonującej projekt:

Pierwszy tester: Daniel Cieślak

Ocena aplikacji po testach:

⇒ Dla osoby znającej kod nie łatwe jest przeprowadzanie testu. Okno logowania zostało sprawdzone kilkukrotnie i nie sprawiło żadnych problemów.
⇒ Później jeszcze raz zostały dokładnie przetestowane opcje w oknie głównym programu. Z racji że struktura w kodzie ma dane typu liczbowego trzeba było sprawdzić co się stanie jeśli użytkownik się pomyli i zamiast nich wpisze litery. Jak się okazało taki błąd nie powoduje zakończenia działania programu. Liczby wynoszą wtedy 0 i można je łatwo zmienić w oknie edycji.
⇒ Reszta opcji w wyniku testów nie sprawiła żadnych problemów. Można by było tylko zmienić nieznacznie wygląd okien dialogowych, ale nie jest to problem.
⇒ Wniosek - aplikacja jest całkiem przejrzysta i działa stabilnie.

4.WNIOSKI

Aplikacja pt. *Rodzinna książka telefoniczna* jest dobrą alternatywą dla osób, które nie posiadają papierowego odpowiednika programu. Dzięki aplikacji użytkownicy mogą dowiedzieć się informacji o swoich krewnych, bliskich, jak i dalekich, jak również przypomnieć sobie o ich istnieniu.

a. Podsumowanie działań

Podsumowując, projekt z przedmiotu Programowanie w języku C2, bazuje na bazie danych zapisanej w pliku tekstowym oraz na strukturze przechowującej osoby z tego pliku. Aplikacja zawiera najważniejsze elementy funkcjonowania, niezbędne do jej poprawnego działania jako książka telefoniczna. Do książki możemy się logować, dodawać i usuwać kontakty, edytować je, wyszukiwać osoby, jak i je sortować. Książka ma również opcje wyświetlania zdjęć kontaktów. Dzięki temu projektowi pogłębiliśmy swoją wiedzę na temat korzystania z biblioteki wxWidgets, jak i, nauczyliśmy się pracować w grupie i wspólnie rozwiązywać zadany problem, co przyda nam się w późniejszych latach.

b. Możliwości rozwoju aplikacji w przyszłości

W przyszłości można by rozwinąć aplikację rodzinnej książki telefonicznej dodając do niej na przykład możliwość wyboru języka, za czym idzie również możliwość utworzenia spisu kontaktów dla różnych państw. Inną rzeczą godną uwagi jest zmiana wyglądu aplikacji, użytkownik mógłby sobie wybierać motyw aplikacji. Dość oczywistą rzeczą jest również dodanie dodatkowych funkcji do projektu jak chociażby możliwość wyświetlania drzewa genealogicznego dla danej rodziny. Z pomocą innych oprogramować do programu można by dodać możliwość czatowania ze sobą osób spośród rodziny czy możliwość dodawania bio do swojej osoby.

5. ZAŁĄCZNIKI

a. Instrukcja instalacji – plik instrukcja_instalacji.pdf
b. Instrukcja uruchomienia aplikacji - plik instrukcja_uruchomienia_aplikacji.pdf
c. Instrukcja obsługi aplikacji – plik instrukcja_obsługi_aplikacji.pdf
d. Wymagania aplikacji – niezbędne oprogramowanie – plik wymagania_aplikacji.pdf
e. Pełen kod źródłowy aplikacji – archiwum Family_Phone_Book.7z